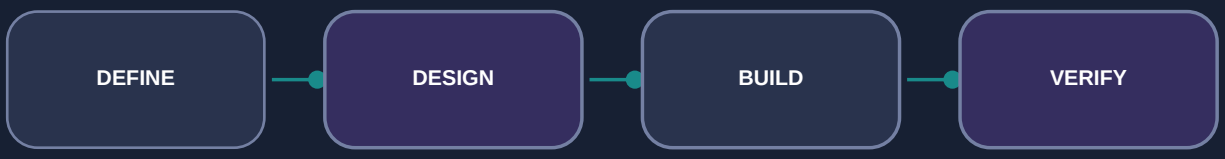
DOCUMENT TRD • Technical Requirements Document

Technical Requirements Document

Architecture, APIs, security controls and operational requirements



Project name: Slotwise (provisional)

Proposed stack: React + Vite + Tailwind • Node.js + Express • Vercel

Public-source clean-room plan • illustrative graphics, not product screenshots

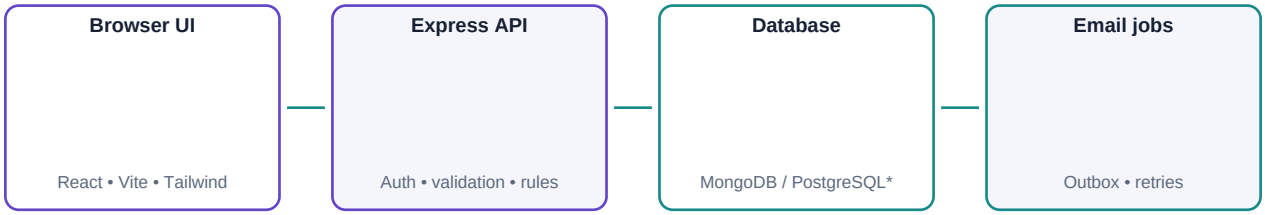
Visual summary

Technical Requirements Document

PURPOSE

Describe the system boundary, backend modules, data decision, security and operational requirements.

KEY VISUAL



***Database decision remains unresolved.**

REVIEW NOTE

Original content is retained after this summary; proposed visuals are illustrative, not real product screenshots.

DOCUMENT 02 OF 6

Technical Requirements Document

Architecture, API contracts, security and operations

Project: Slotwise — provisional independent scheduling product

Reference: Calendly public website and help documentation; no private access.

Platform: Web | **Proposed frontend:** React + Vite + Tailwind CSS | **Backend:** Node.js + Express

Hosting preference: Vercel | **Database:** unresolved conflict between MongoDB preference and required PostgreSQL DDL

Inputs used / assumptions

Stack preference is React/Vite/Tailwind, Node.js/Express, MongoDB and Vercel. The user also required PostgreSQL DDL and hard booking constraints in the planning set. This document treats PostgreSQL as the relational reference design and clearly marks the decision as unresolved. External integrations are deferred unless explicitly approved.

Evidence discipline: Verified public capabilities are distinguished from design proposals and guesses. No review corpus, actual application screenshots, private account data, or production credentials were provided.

1. Architecture overview

Use a modular monolith: one Express application, one approved database, and a durable outbox for email jobs. Vercel can host the frontend and API functions, but long-running jobs must not depend on a persistent in-memory worker.

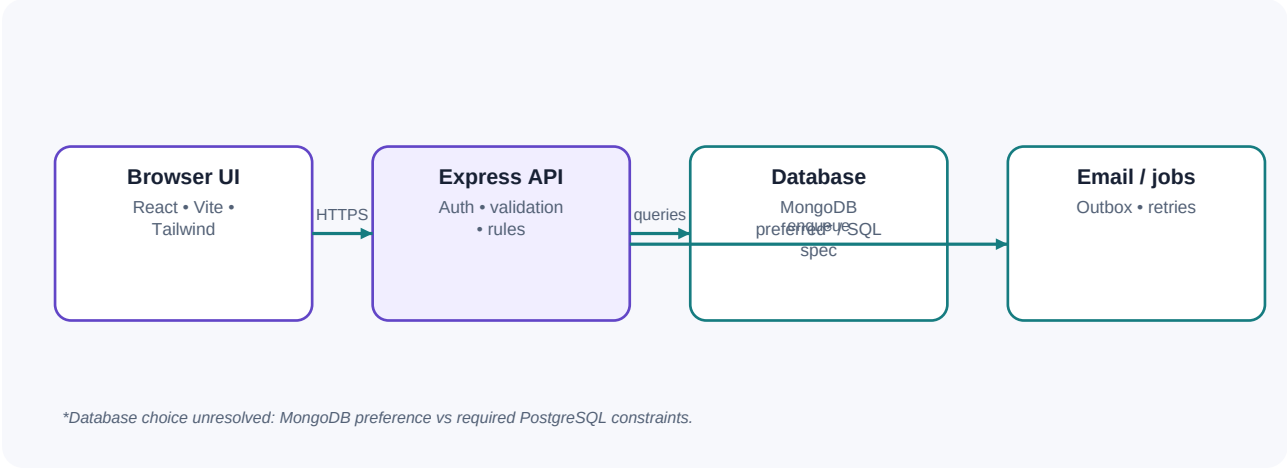


Figure 1. Logical system architecture. The database choice is unresolved; the diagram shows the boundary, not a final vendor commitment.

2. Stack decisions

Layer	Choice	Rationale / status
Frontend	React + Vite	Requested and familiar component architecture
Styling	Tailwind CSS	Responsive tokens and consistent UI
Backend	Node.js + Express	Requested; modular monolith
Database	PostgreSQL (reference) / MongoDB (preference)	Decision gate; PostgreSQL supports required exclusion constraint
ORM	Prisma if PostgreSQL; Mongoose if MongoDB	Choose one based on approved database
Auth	Server-side sessions, HTTP-only cookies	Avoid storing bearer secrets in browser local storage
Email	Resend or equivalent	Transactional confirmations; account must be created by owner
Jobs	Outbox + scheduled worker	Durable retry behavior compatible with serverless deployment
Payments	Stripe Checkout + Customer Portal, optional	Test mode first; no live activation without explicit approval
Hosting	Vercel	Requested; verify function runtime and limits
Monitoring	Structured logs + error monitoring	Operational visibility
Testing	Vitest + Playwright + axe	Unit, integration, E2E, accessibility

3. Frontend architecture

- Route groups: public booking routes (/book/:slug, /booking/confirmed/:reference, /manage/:token) and owner routes (/app/*).
- Shared components: button, input, form error, date picker, time-slot selector, booking summary, status badge, dialog, toast.
- API client centralizes request IDs, typed response handling, and authentication error behavior.
- Data layer exposes stable function signatures so a fake implementation can be replaced with the real API in M5.
- Design tokens and accessible primitives are shared across all screens.

4. Backend modules

Module	Responsibilities	Primary entities
Auth	Registration, verification, login, logout, reset	users, sessions
Events	CRUD, publish/archive, slug validation	event_types
Availability	Weekly rules, local time validation, slot generation	availability_rules
Booking	Slot revalidation, transaction, cancellation/reschedule	bookings
Management	Hashed management tokens and token expiry	management_tokens
Notifications	Outbox, retries, delivery status	outbox_jobs
Operations	Health, request IDs, logs, metrics	Operational logs

5. API contract

Method + path	Access	Input / output	Flow
POST /api/v1/auth/register	Public	email, password → account	F00
POST /api/v1/auth/login	Public	credentials → HTTP-only cookie	F00
POST /api/v1/auth/logout	Owner	session → 204	F00
GET /api/v1/me	Owner	→ profile	F00
GET /api/v1/event-types	Owner	filters → event list	F01
POST /api/v1/event-types	Owner	event fields → event	F01
PATCH /api/v1/event-types/:id	Owner	changes → event	F01
DELETE /api/v1/event-types/:id	Owner	ID → 204/archive	F01
PUT /api/v1/event-types/:id/availability	Owner	weekly rules → saved rules	F01
GET /api/v1/public/:slug	Public	slug → safe event details	F02
GET /api/v1/public/:slug/slots	Public	date range + zone → available slots	F02
POST /api/v1/public/:slug/bookings	Public	slot + invitee + idempotency key → reference	F02
GET /api/v1/bookings	Owner	filters → own bookings	F03
POST /api/v1/manage/:token/cancel	Token holder	→ canceled status	F03
POST /api/v1/manage/:token/reschedule	Token holder	new slot → updated booking	F03
GET /api/v1/health	Monitoring	→ service status	Ops
POST /api/v1/webhooks/stripe	Stripe signature	event → acknowledged	Optional

6. Error response contract

Return a consistent JSON envelope. Do not expose stack traces, internal IDs unnecessarily, SQL details, or secret-bearing values.

```
{ "error": { "code": "SLOT_UNAVAILABLE", "message": "That time is no longer available. Choose another slot.",
"requestId": "req_example", "fieldErrors": {} } }
```

HTTP	Use
400	Malformed request or validation error
401	Missing/invalid session
403	Authenticated but not authorized
404	Unknown or inaccessible resource
409	Slot conflict or idempotency conflict
422	Semantically invalid request
429	Rate limit exceeded
500	Unexpected server error; log request ID only

7. Authentication and authorization

- Use Argon2id or a supported password hashing implementation.
- Verify email before enabling publication.
- Use HTTP-only, Secure (production), appropriately SameSite cookies and CSRF protection where required.
- Reset tokens are random, single-use, expiring and stored hashed.
- Every owner-resource query includes the authenticated owner's ID.
- Test with two owner accounts to verify that cross-owner reads and writes are denied.
- Account deletion revokes sessions, handles future bookings, and deletes or anonymizes personal data according to a documented retention policy.

8. Payments and integrations

Payments are not part of the MVP. If enabled, use Stripe Checkout and Customer Portal, test mode first, signature-verified webhooks, event-ID deduplication, idempotent processing, and subscription state stored in the database. Live mode requires explicit owner approval.

Integration	Minimum access	Review notes
Email provider	Server-side API key	Authenticate sender domain with SPF/DKIM/DMARC
Google Calendar (deferred)	Only scopes needed for availability/event creation	Google OAuth verification and consent review can have lead time
Microsoft calendar (deferred)	Only required calendar/event scopes	Tenant and consent policies vary
Stripe (optional)	Server-side secret and webhook signing secret	No live keys in source control; test mode first

9. Webhooks and jobs

Name	Schedule	Purpose / safeguards
stripe-events	Event-driven, optional	Verify signature; deduplicate event ID
booking-email	Near-real-time after commit	Send confirmation from outbox
email-retry	Every 5 minutes, configurable	Exponential backoff; bounded attempts
stale-hold-cleanup	Every 5 minutes if holds exist	Expire abandoned holds
reminder-email	Optional scheduled job	Respect owner/invitee preferences
data-retention	Daily	Delete expired tokens and eligible records

10. The parts that bite

- **Time zones/DST:** Store instants as UTC; retain IANA zone separately; test ambiguous and nonexistent local times.
- **Idempotency:** Persist request keys and return prior outcomes for safe retries.
- **Race conditions:** Recheck slot in the transaction; enforce no-overlap at the database level.
- **Rate limits:** Throttle login, slot queries and booking submission; honor provider 429 responses.
- **Uploads:** None in MVP. If added, validate type/size, scan as appropriate and use private object storage.
- **Realtime/offline:** Refresh slot data before submission; no realtime requirement initially.
- **Email:** Durable outbox, retry state, bounce monitoring and authenticated domain.
- **Multi-tenancy:** Scope all owner data; never trust user_id supplied by the client.
- **Deletion:** Define data retention, legal record exceptions and deletion propagation.

11. Security and non-functional gates

Control	Requirement
Secrets	Environment variables only; .env.example has names and no values
Input	Server-side schema validation for every endpoint
Authorization	Owner checks on all private resources; second-user test
Webhook	Signature validation and idempotent event processing
Privacy	No personal data in URLs/logs; processor list in privacy policy
Dependencies	Audit and update vulnerable dependencies
Performance	p95 < 500 ms for ordinary reads under defined load
Accessibility	WCAG 2.2 AA target and keyboard core flow
Backups	Automated daily backups and tested restore
Observability	Structured logs, request IDs, health endpoint and alerting

12. Deployment architecture

Deploy the static frontend and API to Vercel where compatible with runtime constraints. Use a separately managed production database, durable scheduled job invocation, environment-specific secrets, verified email DNS and monitoring. Confirm Vercel function limits before selecting a job architecture.

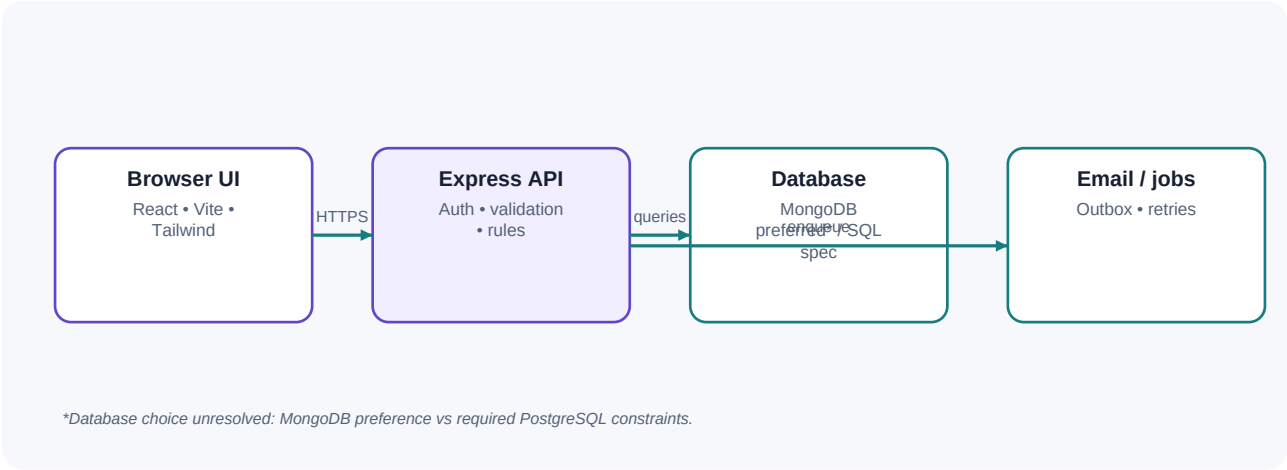


Figure 2. Deployment boundary. The actual provider configuration remains to be implemented and tested.

13. Open architecture decisions

- Approve PostgreSQL or retain MongoDB and redesign the no-overlap invariant.
- Choose email provider and create sender domain.
- Decide whether OAuth, payments and reminders are required for initial release.
- Select session store and hosting configuration based on Vercel runtime limits.
- Define target load and backup recovery objectives.